

Histogrammar basic tutorial

Histogrammar is a Python package that allows you to make histograms from numpy arrays, and pandas and spark dataframes. (There is also a scala backend for Histogrammar.)

This basic tutorial shows how to:

- make histograms with numpy arrays and pandas dataframes,
- plot them,
- make multi-dimensional histograms,
- the various histogram types,
- to make many histograms at ones,
- and store and retrieve them.

Enjoy!

```
In [1]: %%capture
# install histogrammar (if not installed yet)

!"{sys.executable}" -m pip install histogrammar
```

```
In [2]: import histogrammar as hg
```

```
In [3]: import numpy as np
import pandas as pd
```

Data generation

Let's first load some data!

```
In [4]: # open a pandas dataframe for use below
from histogrammar import resources

df = pd.read_csv(resources.data("test.csv.gz"), parse_dates=["date"])
```

```
In [5]: df.head()
```

Out[5]:

		_id	address	age	balance	company	email	eyeColor	favoriteFruit	gender	guid	...	isActive	latitude	longitude	name
0	565ebe61ffa6f757b5029f29	109 Monument Walk, Vicksburg, California, 6247	16	\$1,333.24	DOGSPA	tamekajoyce@dogspa.com	grey	banana	female	c06a96e4-31e2-426e-aa5a-51e3c71991aa	...	False	18.821230	-131.721235	Tameka Joyce	
1	565ebe616e073b23430d7052	602 Harkness Avenue, Tyhee, Arizona, 5853	57	\$3,777.44	APPLICA	rosettaflynn@applica.com	brown	banana	female	6bbcaf11-a6b9-4f00-9db1-84fc11161526	...	True	-82.134610	52.720886	Rosetta Flynn	
2	565ebe6141fdb23a67dd7584	286 Fay Court, Forbestown, Pennsylvania, 6141	26	\$1,248.97	HOMETOWN	kathiemurray@hometown.com	blue	strawberry	female	b85ef702-39f4-42dd-a87f-ce97864c3551	...	True	62.383017	-85.223346	Kathie Murray	
3	565ebe61c33636f6df907b06	617 Baycliff Terrace, Castleton, Oklahoma, 7714	63	\$1,049.77	ISIS	bennetthahn@isis.com	green	strawberry	male	b42f1df9-839f-4f39-b3de-7ea3ac177786	...	True	-26.913326	-67.825793	Bennett Hahn	
4	565ebe61225511fd3288bb63	262 Scholes Street, Fillmore, Utah, 2905	74	\$2,942.64	FUTURITY	cottoncarson@futuraity.com	red	strawberry	male	14e518aa-2701-4397-94e7-72b9c27655ad	...	False	-2.777228	155.341981	Cotton Carson	

5 rows × 21 columns

Let's fill a histogram!

Histogrammar treats histograms as objects. You will see this has various advantages.

Let's fill a simple histogram with a numpy array.

In [6]:

```
# this creates a histogram with 100 even-sized bins in the (closed) range [-5, 5]
hist1 = hg.Bin(num=100, low=-5, high=5)
```

In [7]:

```
# filling it with one data point:
hist1.fill(0.5)
```

In [8]:

```
hist1.entries
```

Out[8]: 1.0

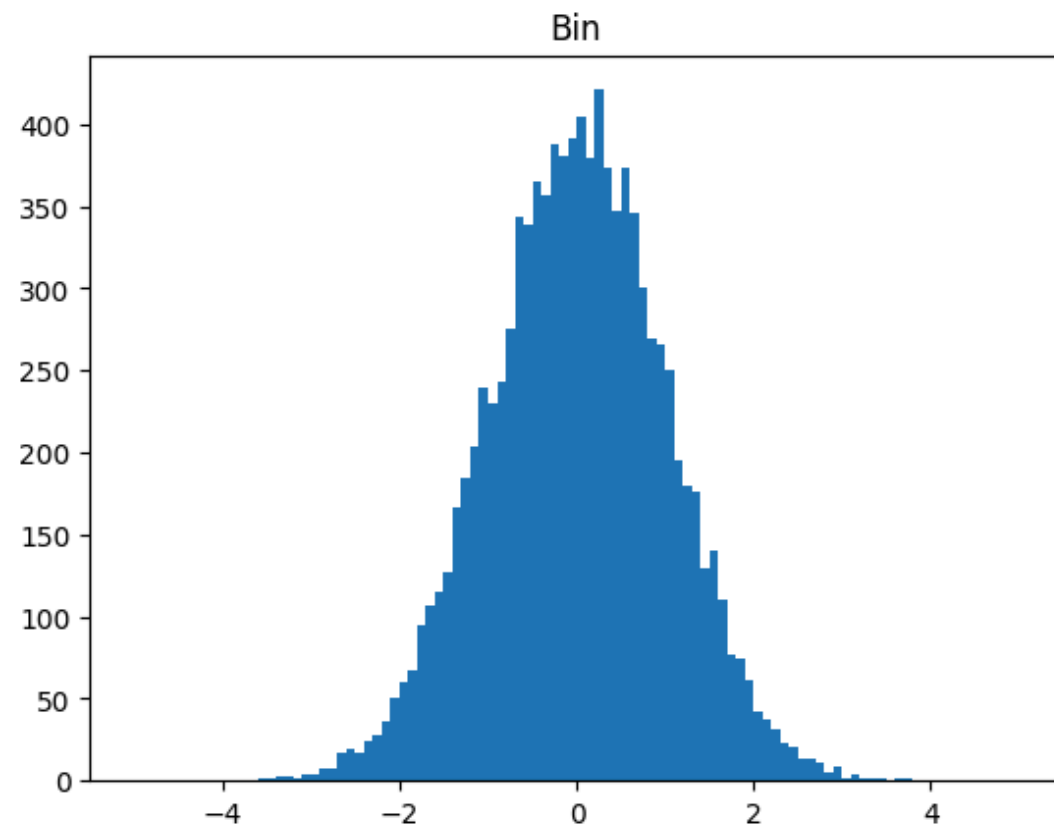
In [9]:

```
# filling the histogram with an array:
hist1.fill(numpy(np.random.normal(size=10000)))
```

```
In [10]: hist1.entries
```

```
Out[10]: 10001.0
```

```
In [11]: # let's plot it  
hist1.plot.matplotlib();
```



```
In [12]: # Alternatively, you can call this to make the same histogram:  
# hist1 = hg.Histogram(num=100, low=-5, high=5)
```

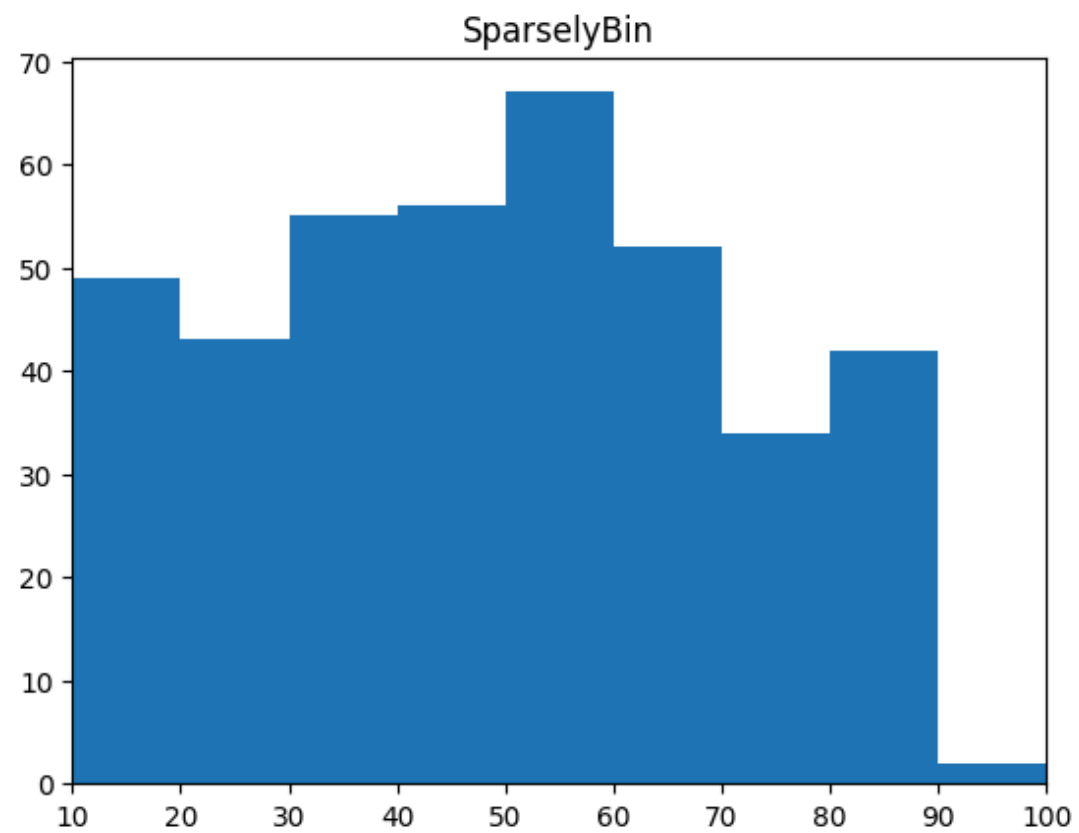
Histogrammar also supports open-ended histograms, which are sparsely represented. Open-ended histograms are used when you have a distribution of known scale (bin width) but unknown domain (lowest and highest bin index). Bins in a sparse histogram only get created and filled if the corresponding data points are encountered.

A sparse histogram has a `binWidth`, and optionally an `origin` parameter. The `origin` is the left edge of the bin whose index is 0 and is set to 0.0 by default. Sparse histograms are nice if you don't want to restrict the range, for example for tracking data distributions over time, which may have large, sudden outliers.

```
In [13]: hist2 = hg.SparselyBin(binWidth=10, origin=0)
```

```
In [14]: hist2.fill.numpy(df['age'].values)
```

```
In [15]: hist2.plot.matplotlib();
```

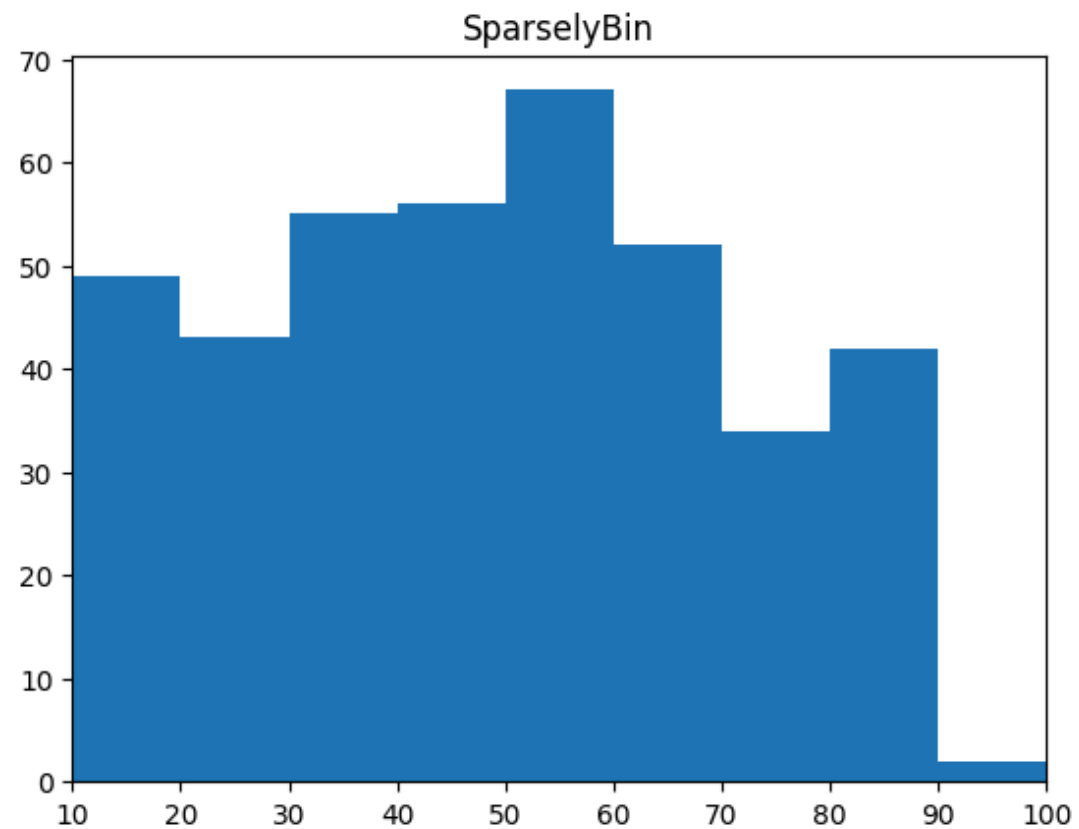


```
In [16]: # Alternatively, you can call this to make the same histogram:  
# hist2 = hg.SparselyHistogram(binWidth=10)
```

Filling from a dataframe

Let's make the same 1d (sparse) histogram directly from a (pandas) dataframe.

```
In [17]: hist3 = hg.SparselyBin(binWidth=10, origin=0, quantity='age')  
hist3.fill.numpy(df)  
hist3.plot.matplotlib();
```



When importing histogrammar, pandas (and spark) dataframes get extra functions to create histograms that all start with "hg_". For example: hg_Bin or hg_SparselyBin. Note that the column "age" is picked by setting quantity="age", and also that the filling step is done automatically.

```
In [18]: # Alternatively, do:
hist3 = df.hg_SparselyBin(binWidth=10, origin=0, quantity='age')

# ... where hist3 automatically picks up column age from the dataframe,
# ... and does not need to be filled by calling fill.numpy() explicitly.
```

Handy histogram methods

For any 1-dimensional histogram extract the bin entries, edges and centers as follows:

```
In [19]: # full range of bin entries, and those in a specified range:
(hist3.bin_entries(), hist3.bin_entries(low=30, high=80))
```

```
Out[19]: (array([49., 43., 55., 56., 67., 52., 34., 42., 2.]),
          array([55., 56., 67., 52., 34.]))
```

```
In [20]: # full range of bin edges, and those in a specified range:
(hist3.bin_edges(), hist3.bin_edges(low=31, high=71))
```

```
Out[20]: (array([ 10., 20., 30., 40., 50., 60., 70., 80., 90., 100.]),
          array([30., 40., 50., 60., 70., 80.]))
```

```
In [21]: # full range of bin centers, and those in a specified range:
(hist3.bin_centers(), hist3.bin_centers(low=31, high=80))
```

```
Out[21]: (array([15., 25., 35., 45., 55., 65., 75., 85., 95.]),  
         array([35., 45., 55., 65., 75.]))
```

```
In [22]: hsum = hist2 + hist3  
hsum.entries
```

```
Out[22]: 800.0
```

```
In [23]: hsum *= 4  
hsum.entries
```

```
Out[23]: 3200.0
```

Irregular bin histogram variants

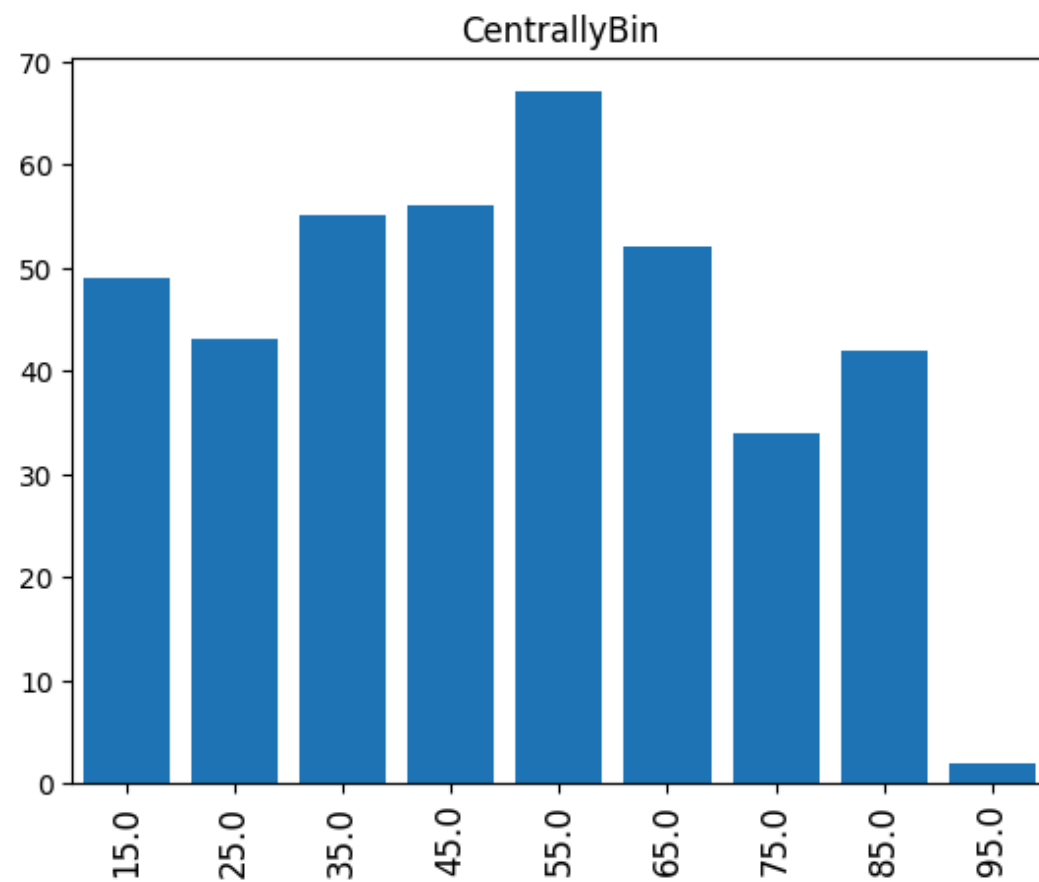
There are two other open-ended histogram variants in addition to the `SparselyBin` we have seen before. Whereas `SparselyBin` is used when bins have equal width, the others offer similar alternatives to a single fixed bin width.

There are two ways:

- `CentrallyBin` histograms, defined by specifying bin centers;
- `IrregularlyBin` histograms, with irregular bin edges.

They both partition a space into irregular subdomains with no gaps and no overlaps.

```
In [24]: hist4 = hg.CentrallyBin(centers=[15, 25, 35, 45, 55, 65, 75, 85, 95], quantity='age')  
hist4.fill.numpy(df)  
hist4.plot.matplotlib();
```

```
In [25]: hist4.bin_edges()
```

```
Out[25]: array([-inf, 20., 30., 40., 50., 60., 70., 80., 90., inf])
```

Note the slightly different plotting style for CentrallyBin histograms (e.g. x-axis labels are central values instead of edges).

Multi-dimensional histograms

Let's make a multi-dimensional histogram. In Histogrammar, a multi-dimensional histogram is composed as two recursive histograms.

We will use histograms with irregular binning in this example.

```
In [26]: edges1 = [-100, -75, -50, -25, 0, 25, 50, 75, 100]
edges2 = [-200, -150, -100, -50, 0, 50, 100, 150, 200]
```

```
In [27]: hist1 = hg.IrregularlyBin(edges=edges1, quantity='latitude')
hist2 = hg.IrregularlyBin(edges=edges2, quantity='longitude', value=hist1)

# for 3 dimensions or higher simply add the 2-dim histogram to the value argument
hist3 = hg.SparselyBin(binWidth=10, quantity='age', value=hist2)
```

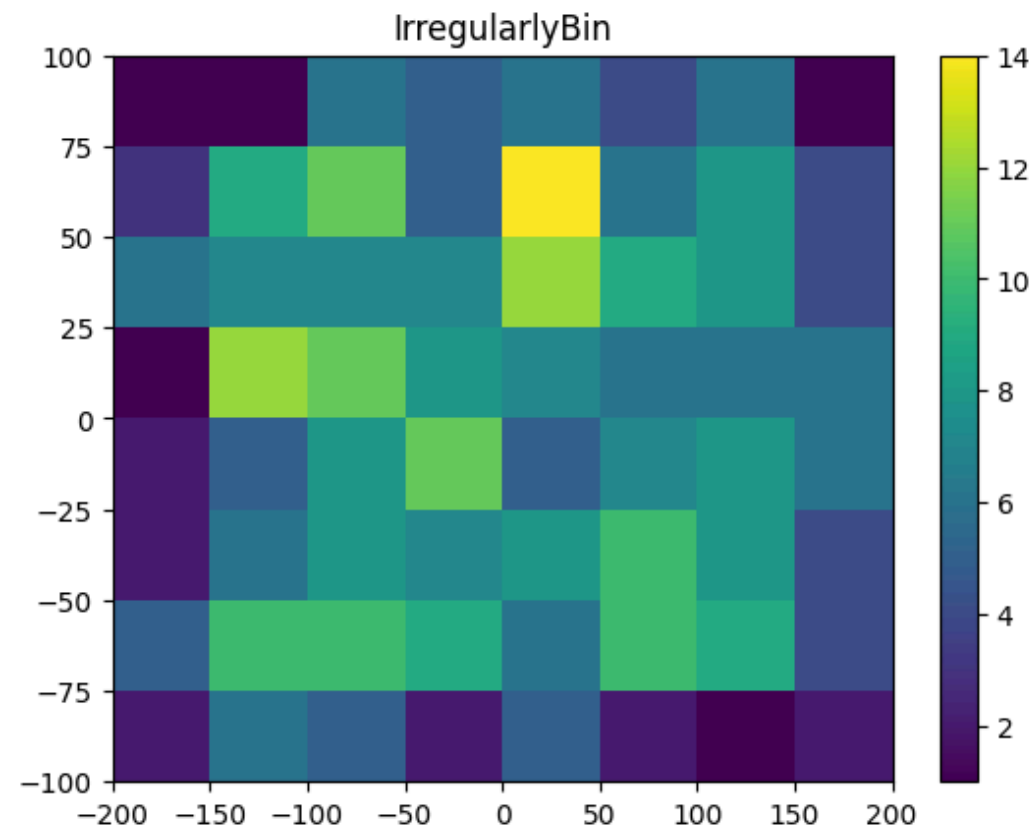
```
In [28]: hist1.bin_centers()
```

```
Out[28]: array([-inf, -87.5, -62.5, -37.5, -12.5, 12.5, 37.5, 62.5, 87.5,
               inf])
```

```
In [29]: hist2.bin_centers()
```

```
Out[29]: array([-inf, -175., -125., -75., -25.,  25.,  75., 125., 175.,  
              inf])
```

```
In [30]: hist2.fill.numpy(df)  
hist2.plot.matplotlib();
```



```
In [31]: # number of dimensions per histogram  
(hist1.n_dim, hist2.n_dim, hist3.n_dim)
```

```
Out[31]: (1, 2, 3)
```

Accessing bin entries

For most 2+ dimensional histograms, one can get the bin entries and centers as follows:

```
In [32]: from histogrammar.plot.hist_numpy import get_2dgrid  
  
x_labels, y_labels, grid = get_2dgrid(hist2)
```

```
In [33]: y_labels, grid
```



```
Out[33]: (['-inf',
'-87.5',
'-62.5',
'-37.5',
'-12.5',
'12.5',
'37.5',
'62.5',
'87.5',
'inf'],
array([[ 0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.],
[ 0.,  2.,  6.,  5.,  2.,  5.,  2.,  1.,  2.,  0.],
[ 0.,  5., 10., 10.,  9.,  6., 10.,  9.,  4.,  0.],
[ 0.,  2.,  6.,  8.,  7.,  8., 10.,  8.,  4.,  0.],
[ 0.,  2.,  5.,  8., 11.,  5.,  7.,  8.,  6.,  0.],
[ 0.,  1., 12., 11.,  8.,  7.,  6.,  6.,  6.,  0.],
[ 0.,  6.,  7.,  7.,  7., 12.,  9.,  8.,  4.,  0.],
[ 0.,  3.,  9., 11.,  5., 14.,  6.,  8.,  4.,  0.],
[ 0.,  1.,  1.,  6.,  5.,  6.,  4.,  6.,  1.,  0.],
[ 0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.]])
```

Accessing a sub-histogram

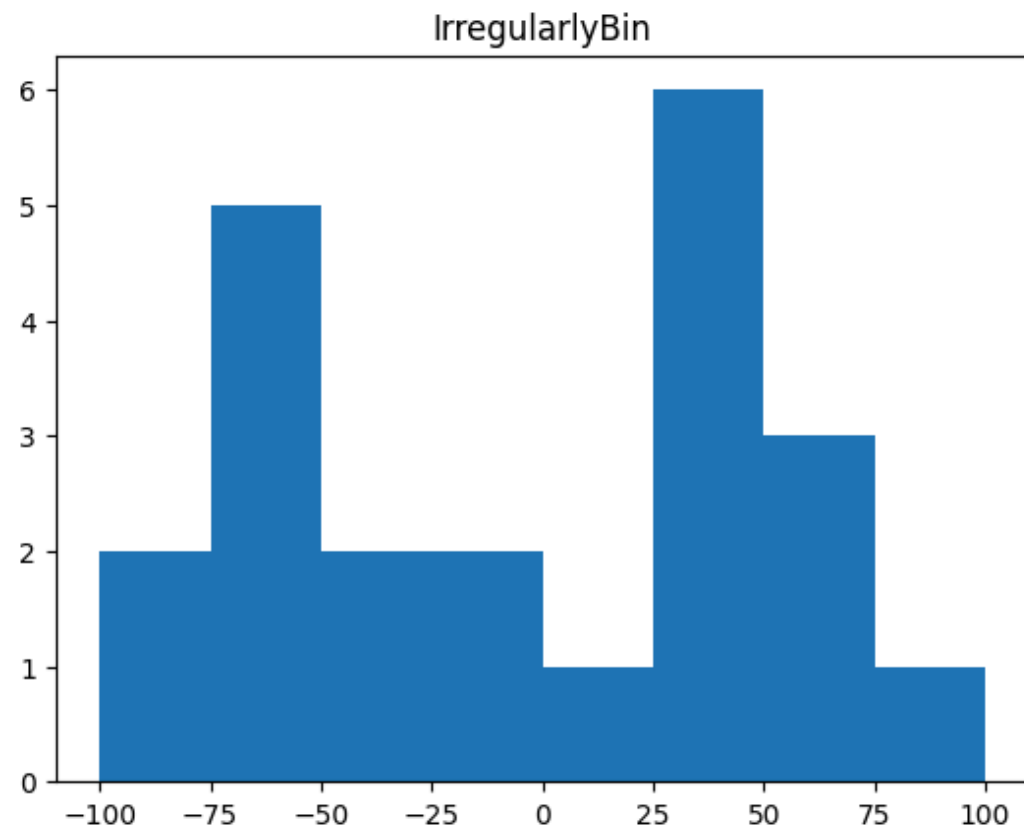
Depending on the histogram type of the first axis, hg.Bin or other, one can access the sub-histograms directly from: hist.values or hist.bins

```
In [34]: # Acces sub-histograms from IrregularlyBin from hist.bins
# The first item of the tuple is the lower bin-edge of the bin.
hist2.bins[1]
```

```
Out[34]: (-200.0,
<IrregularlyBin values=Count thresholds=(-inf, -100.0, -75.0, -50.0, -25.0, 0.0, 25.0, 50.0, 75.0, 100.0) nanflow=Count>)
```

```
In [35]: h = hist2.bins[1][1]
h.plot.matplotlib()
```

```
Out[35]: <Axes: title={'center': 'IrregularlyBin'}>
```



```
In [36]: h.bin_entries()
```

```
Out[36]: array([0., 2., 5., 2., 2., 1., 6., 3., 1., 0.])
```

Histogram types recap

So far we have covered the histogram types:

- Bin histograms: with a fixed range and even-sized bins,
- SparselyBin histograms: open-ended and with a fixed bin-width,
- CentrallyBin histograms: open-ended and using bin centers.
- IrregularlyBin histograms: open-ended and using (irregular) bin edges,

All of these process numeric variables only.

Categorical variables

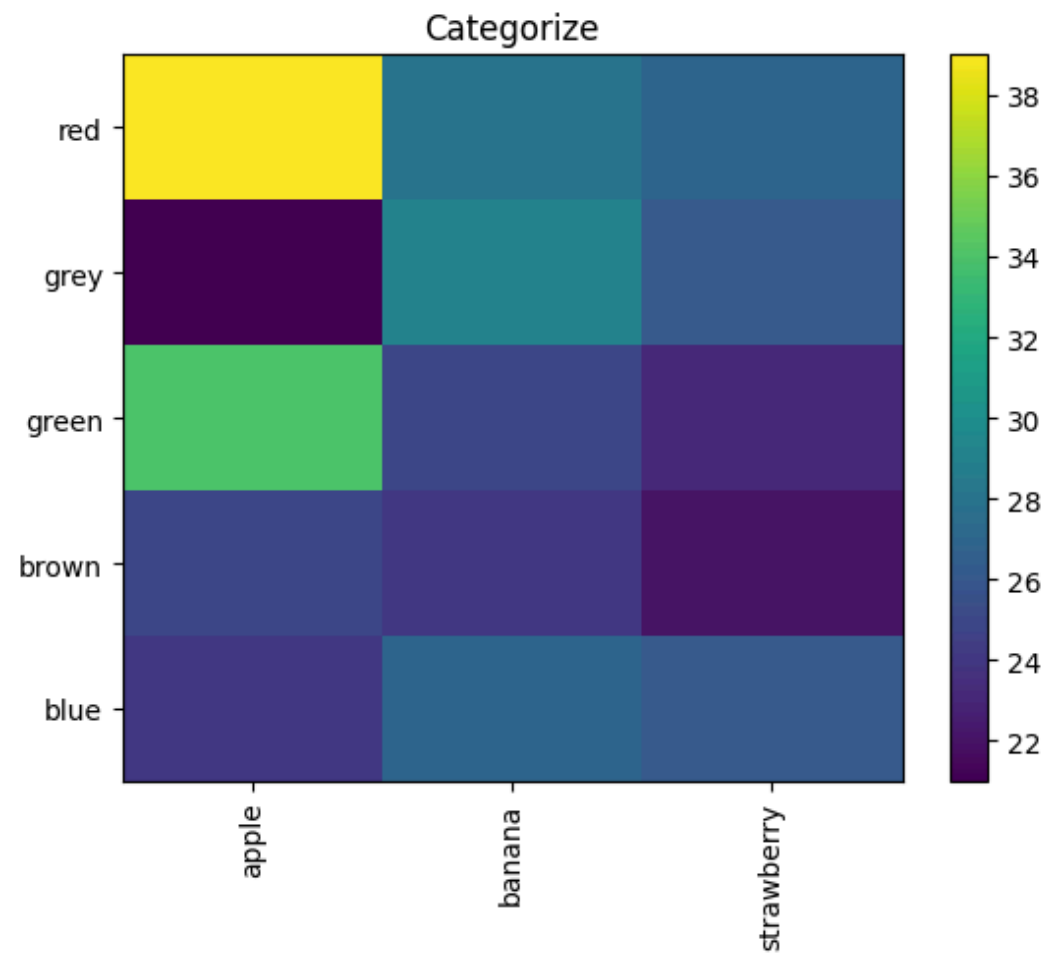
For categorical variables use the Categorize histogram

- Categorize histograms: accepting categorical variables such as strings and booleans.

```
In [37]: histy = hg.Categorize('eyeColor')
histx = hg.Categorize('favoriteFruit', value=histy)
```

```
In [38]: histx.fill.numpy(df)
```

```
histx.plot.matplotlib();
```

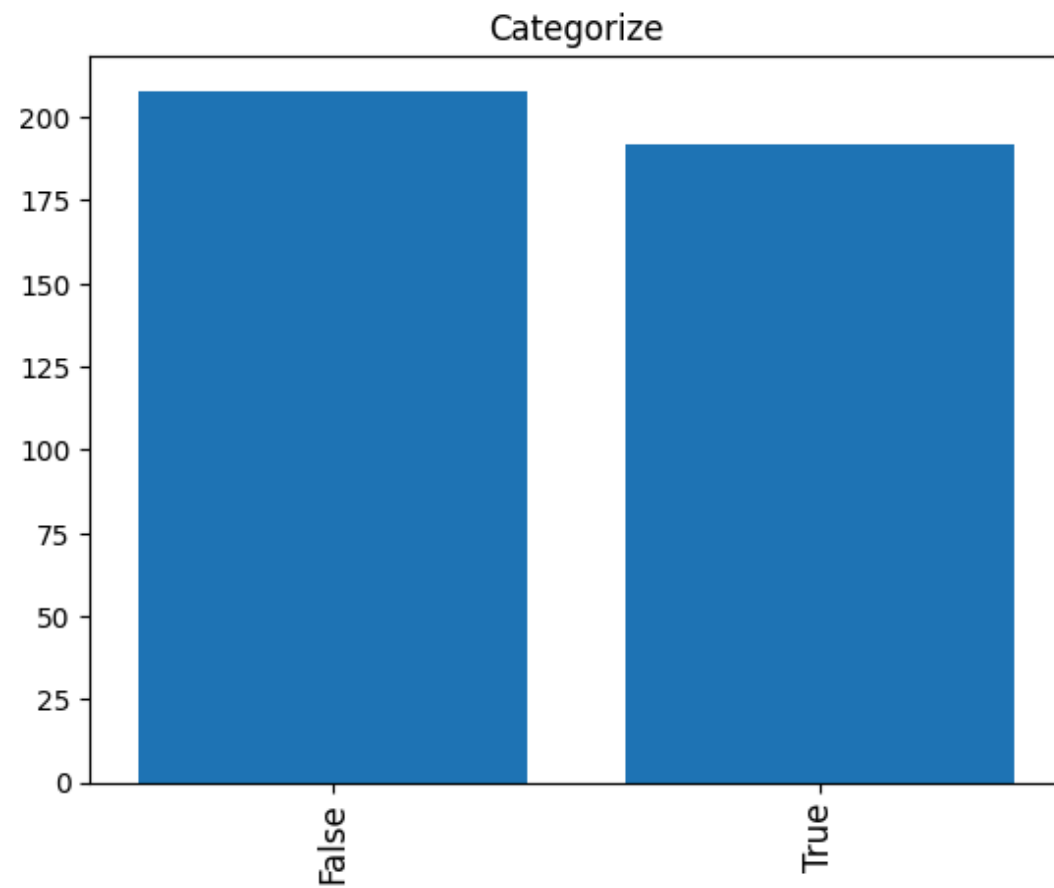


```
In [39]: # show the datatype(s) of the histogram
histx.datatype
```

```
Out[39]: [str, str]
```

Categorize histograms also accept booleans:

```
In [40]: histy = hg.Categorize('isActive')
histy.fill.numpy(df)
histy.plot.matplotlib();
```



```
In [41]: histy.bin_entries()
```

```
Out[41]: array([208., 192.])
```

```
In [42]: histy.bin_labels()  
# histy.bin_centers() will work as well for Categorize histograms
```

```
Out[42]: array([False,  True])
```

Other histogram functionality

There are several more histogram types:

- Minimize, Maximize: keep track of the min or max value of a numeric distribution,
- Average, Deviate: keep track of the mean or mean and standard deviation of a numeric distribution,
- Sum: keep track of the sum of a numeric distribution,
- Stack: keep track how many data points pass certain thresholds.
- Bag: works like a dict, it keeps tracks of all unique values encountered in a column, and can also do this for vector s of numbers. For strings, Bag works just like the Categorize histogram.

```
In [43]: hmin = df.hg_Minimize('latitude')  
hmax = df.hg_Maximize('longitude')  
(hmin.min, hmax.max)
```

```
Out[43]: (-88.814195, 178.773393)
```

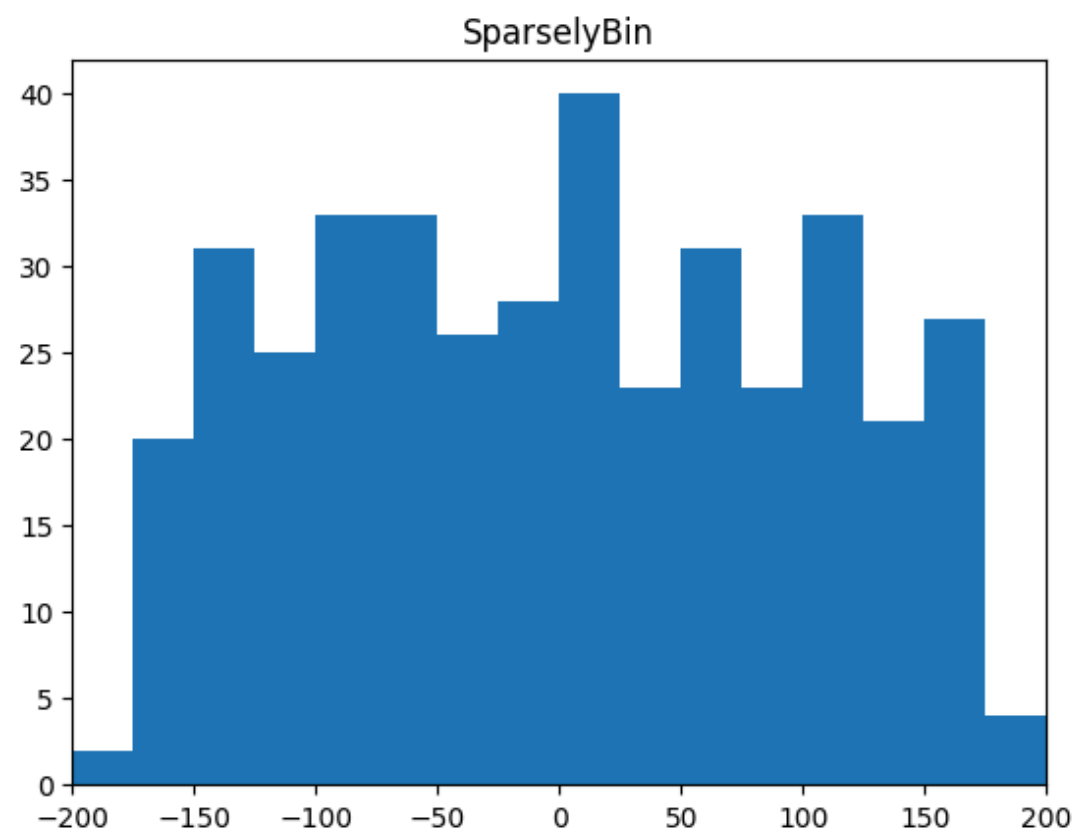
```
In [44]: havg = df.hg_Average('latitude')
hdev = df.hg_Deviate('longitude')
(havg.mean, hdev.mean, hdev.variance)
```

```
Out[44]: (1.574939025, -0.3369409250000002, 9785.23543919839)
```

```
In [45]: hsum = df.hg_Sum('age')
hsum.sum
```

```
Out[45]: 19396.0
```

```
In [46]: # let's illustrate the Stack histogram with longitude distribution
# first we plot the regular distribution
hl = df.hg_SparselyBin(25, 'longitude')
hl.plot.matplotlib();
```



```
In [47]: # Stack counts how often data points are greater or equal to the provided thresholds
thresholds = [-200, -150, -100, -50, 0, 50, 100, 150, 200]
```

```
In [48]: hs = df.hg_Stack(thresholds=thresholds, quantity='longitude')
hs.thresholds
```

```
Out[48]: [-inf, -200.0, -150.0, -100.0, -50.0, 0.0, 50.0, 100.0, 150.0, 200.0]
```

```
In [49]: hs.bin_entries()
```

```
Out[49]: array([400., 400., 378., 322., 256., 202., 139., 85., 31., 0.])
```

Stack histograms are useful to make efficiency curves.

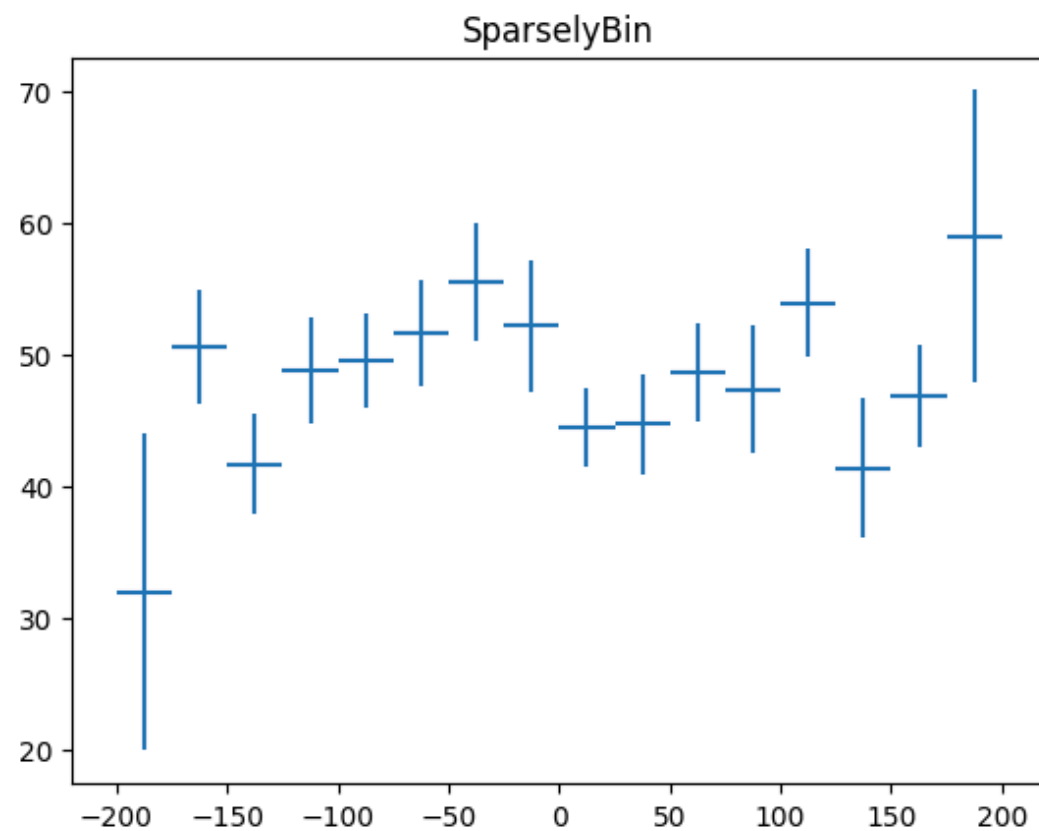
With all these histograms you can make multi-dimensional histograms. For example, you can evaluate the mean and standard deviation of one feature as a function of bins of another feature. (A "profile" plot, similar to a box plot.)

```
In [50]: hav = hg.Deviate('age')
hlo = hg.SparselyBin(25, 'longitude', value=hav)
hlo.fill.numpy(df)
```

```
In [51]: hlo.bins
```

```
Out[51]: {np.int64(-8): <Deviate mean=32.0 variance=289.0>,
 np.int64(-7): <Deviate mean=50.6 variance=384.14000000000016>,
 np.int64(-6): <Deviate mean=41.70967741935484 variance=458.72216441207064>,
 np.int64(-5): <Deviate mean=48.8 variance=409.99999999999997>,
 np.int64(-4): <Deviate mean=49.60606060606061 variance=425.8751147842056>,
 np.int64(-3): <Deviate mean=51.666666666666664 variance=529.55555555555554>,
 np.int64(-2): <Deviate mean=55.53846153846154 variance=519.2485207100589>,
 np.int64(-1): <Deviate mean=52.17857142857143 variance=707.2181122448981>,
 np.int64(0): <Deviate mean=44.525 variance=363.24937500000016>,
 np.int64(1): <Deviate mean=44.73913043478261 variance=335.4971644612477>,
 np.int64(2): <Deviate mean=48.70967741935484 variance=435.62539021852245>,
 np.int64(3): <Deviate mean=47.34782608695652 variance=542.4877126654065>,
 np.int64(4): <Deviate mean=53.90909090909091 variance=547.2947658402202>,
 np.int64(5): <Deviate mean=41.42857142857143 variance=577.2925170068027>,
 np.int64(6): <Deviate mean=46.888888888888886 variance=409.7283950617286>,
 np.int64(7): <Deviate mean=59.0 variance=489.0>}
```

```
In [52]: hlo.plot.matplotlib();
```

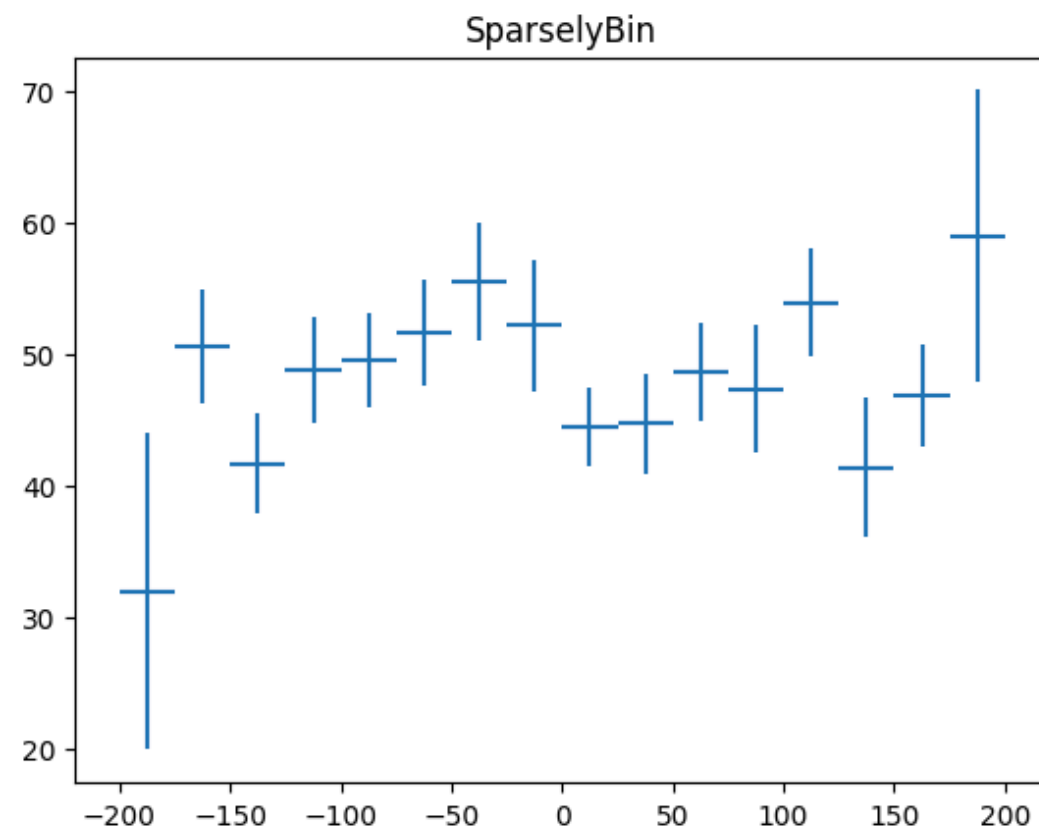


Convenience functions

There are several convenience functions to make such composed histograms. These are:

- `Profile`: Convenience function for creating binwise averages.
- `SparselyProfile`: Convenience function for creating sparsely binned binwise averages.
- `ProfileErr`: Convenience function for creating binwise averages and variances.
- `SparselyProfile`: Convenience function for creating sparsely binned binwise averages and variances.
- `TwoDimensionallyHistogram`: Convenience function for creating a conventional, two-dimensional histogram.
- `TwoDimensionallySparselyHistogram`: Convenience function for creating a sparsely binned, two-dimensional histogram.

```
In [53]: # For example, call this convenience function to make the same histogram as above:  
hlo = df.hg_SparselyProfileErr(25, 'longitude', 'age')  
hlo.plot.matplotlib();
```



Overview of histograms

Here you can find the list of all available histograms and aggregators and how to use each one:

<https://histogrammar.github.io/histogrammar-docs/specification/1.0/>

The most useful aggregators are the following. Tinker with them to get familiar; building up an analysis is easier when you know "there's an app for that."

Simple counters:

- `Count` : just counts. Every aggregator has an `entries` field, but `Count` only has this field.

- `Average` and `Deviate` : add mean and variance, cumulatively.
- `Minimize` and `Maximize` : lowest and highest value seen.

Histogram-like objects:

- `Bin` and `SparselyBin` : split a numerical domain into uniform bins and redirect aggregation into those bins.
- `Categorize` : split a string-valued domain by unique values; good for making bar charts (which are histograms with a string-valued axis).
- `CentrallyBin` and `IrregularlyBin` : split a numerical domain into arbitrary subintervals, usually for separate plots like particle pseudorapidity or collision centrality.

Collections:

- `Label` , `UntypedLabel` , and `Index` : bundle objects with string-based keys (`Label` and `UntypedLabel`) or simply an ordered array (effectively, integer-based keys) consisting of a single type (`Label` and `Index`) or any types (`UntypedLabel`).
- `Branch` : for the fourth case, an ordered array of any types. A `Branch` is useful as a "cable splitter". For instance, to make a histogram that tracks minimum and maximum value, do this:

Making many histograms at once

There a nice method to make many histograms in one go. See [here](#).

By default automagical binning is applied to make the histograms.

More details one how to use this function are found in in the advanced tutorial.

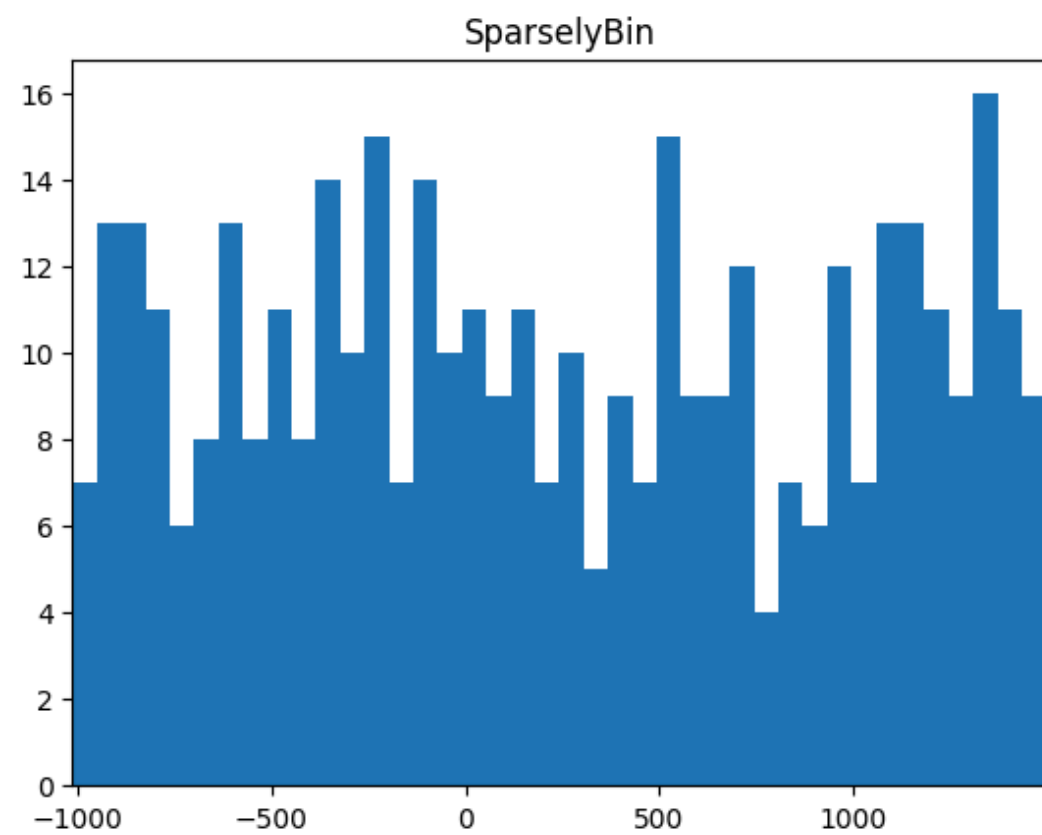
```
In [54]: hists = df.hg_make_histograms()

0%|          | 0/21 [00:00<?, ?it/s]
100%|██████████| 21/21 [00:00<00:00, 2844.42it/s]

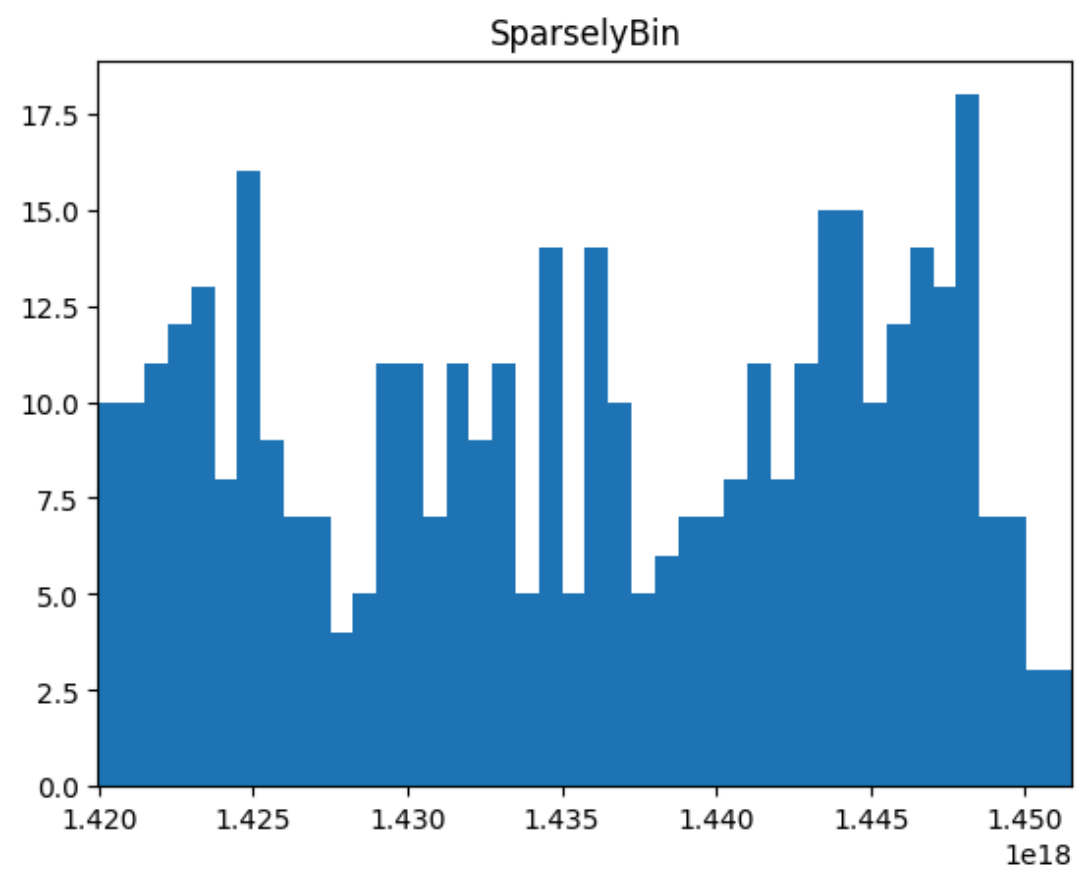
In [55]: hists.keys()

Out[55]: dict_keys(['_id', 'address', 'age', 'balance', 'company', 'currency', 'date', 'email', 'eyeColor', 'favoriteFruit', 'gender', 'guid', 'index', 'isActive', 'latitude', 'longitude', 'name', 'phone', 'registered', 'tags', 'transaction'])

In [56]: h = hists['transaction']
h.plot.matplotlib();
```

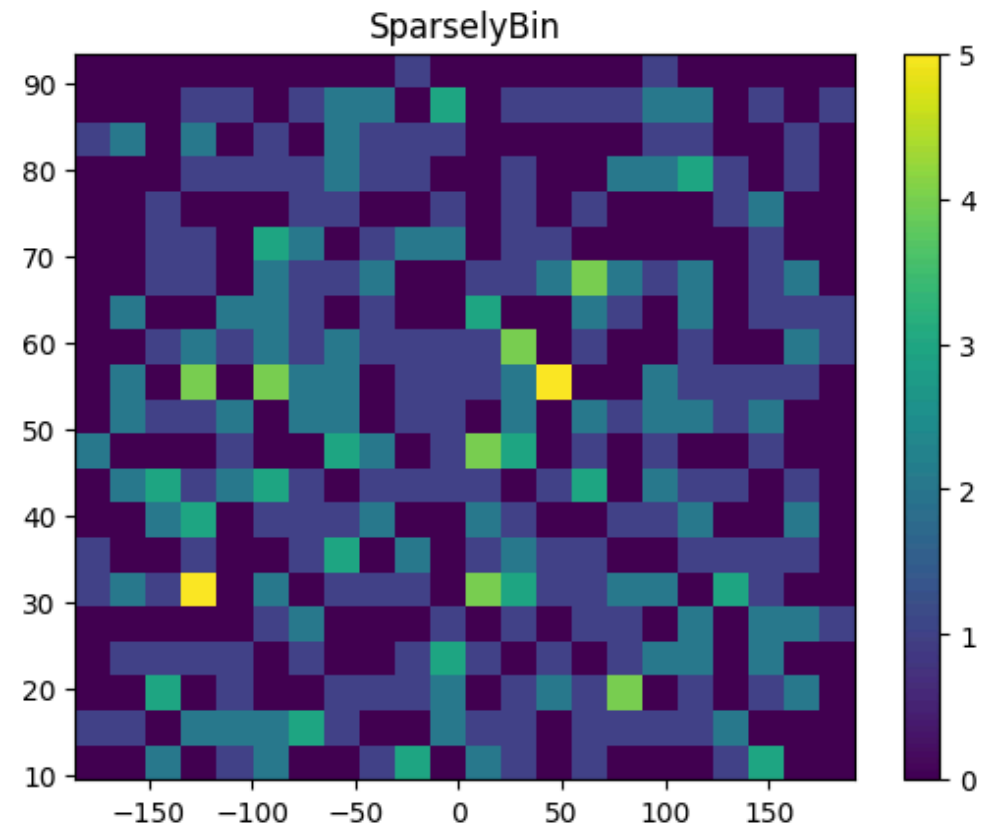
```
In [57]: h = hists['date']  
h.plot.matplotlib();
```



```
In [58]: # you can also select which and make multi-dimensional histograms
hist = df.hg_make_histograms(features = ['longitude:age'])
```

```
0%|          | 0/1 [00:00<?, ?it/s]
100%|██████████| 1/1 [00:00<00:00, 475.22it/s]
```

```
In [59]: hist = hist['longitude:age']
hist.plot.matplotlib();
```

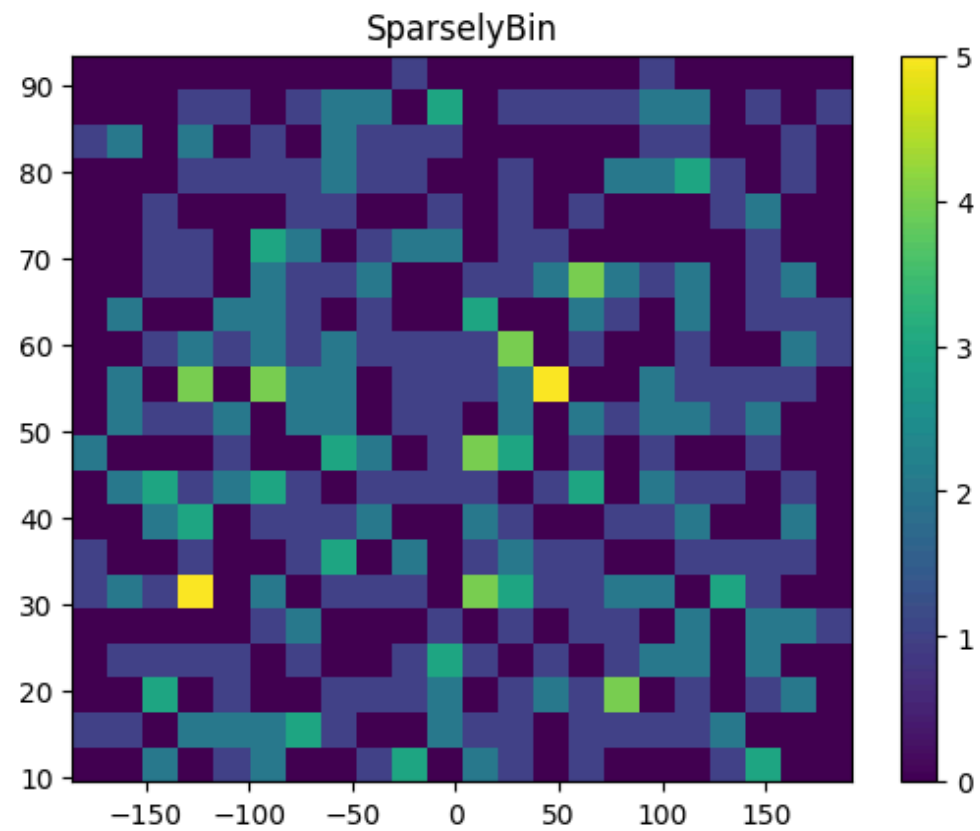


Storage

Histograms can be easily stored and retrieved in/from the json format.

```
In [60]: # storage
hist.toJsonFile('long_age.json')
```

```
In [61]: # retrieval
factory = hg.Factory()
hist2 = factory.fromJsonFile('long_age.json')
hist2.plot.matplotlib();
```



```
In [62]: # we can store the histograms if we want to
import json

from histogrammar.util import dumper

# store
with open('histograms.json', 'w') as outfile:
    json.dump(hists, outfile, default=dumper)

# and load again
with open('histograms.json') as handle:
    hists2 = json.load(handle)
```

```
In [63]: hists.keys()
```

```
Out[63]: dict_keys(['longitude:age'])
```

Advanced tutorial

The [advanced tutorial](#) shows:

- How to work with spark dataframes.
- More details on this nice method to make many histograms in one go. For example how to set bin specifications.